

TEKNİK DOKÜMANTASYON VE KULLANIM REHBERİ

EXCEL VBA VAKA RAPORLAMA UYGULAMASI

Bu çalışma, **Vaka Raporlama Uygulaması** projesinin kod yapısını, modüler mimarisini ve sistem bakım detaylarını aktaran teknik kullanım rehberidir.

Son Rapor Güncelleme Tarihi

📅 4 Ağustos 2026

Uygulama Kaynak Kodları

🔗 GitHub Repository



İçindekiler

1	Proje Hakkında	2
1.1	Projenin Amacı	2
1.2	Öne Çıkan Özellikler	2
2	Kullanıcı İçin Tavsiyeler	3
3	Sorun Giderme	4
4	Uygulama Onarım Rehberi	6
4.1	Sayfa ve Panel Onarım Rehberi	6
4.2	Tarih ve Dönem Yönetimi	6
4.3	Veritabanı Sayfa Yapıları ve Görsel Referanslar	7
5	Proje Dosya Yapısı ve Mimari	8
6	VBA Nedir ve Temel Kavramlar	9
6.1	VBA (Visual Basic for Applications) Nedir?	9
6.2	Excel VBA Temel Bileşenleri	9
6.3	VBA Programlama Mantığı ve Temel Yapılar	10
6.4	Projeden Örnek Kod Bloğu ve İncelemesi	11
7	Detaylı Modül İncelemeleri	12
7.1	ThisWorkbook	12
7.2	clsDynamicButton	13
7.3	mod_Config	14
7.4	mod_UI_Manager	15
7.5	mod_UI_Events	17
7.6	mod_UI_View	19
7.7	mod_Services	21
8	Geliştirici Rehberi	24
8.1	Senaryo 1: Gelen Rapor Biçimi veya Sütun Yapısı Değişirse	24
8.2	Senaryo 2: Arayüze Yeni Bir Panel Ekleme (CreateYEPYENIPanel)	25
8.3	Senaryo 3: Yeni Bir İş Mantığı veya Hesaplama Servisi Ekleme	26
8.4	Senaryo 4: Yeni Veritabanı Çalışma Sayfası Ekleme ve Geliştirme Kuralları	27
9	Sonuç	28

Proje Hakkında



Hızlı Raporlama

Saniyeler İçinde Analiz



%0 Hata Oranı

Otomatik Veri Kontrolü



Modüler Yapı

OOP & Katmanlı Mimari

1.1 Projenin Amacı

Bu otomasyon projesi, Kurum bünyesinde yürütülen **Arıza** ve **Bakım** süreçlerinin takibini otomatize etmek, manuel veri girişinden kaynaklanan **insani hataları en aza indirmek** ve raporlama süreçlerini saniyeler seviyesine indirmek amacıyla geliştirilmiştir.

💡 Projenin Temel Hedefi

Karmaşık Excel sayfaları ve hücre haritaları arasında kaybolmak yerine; kullanıcıya modern, dinamik ve modüler bir panel arayüzü sunarak iş akışlarını uçtan uca standardize etmektir.

1.2 Öne Çıkan Özellikler



Dinamik Panel Arayüzü: Form karmaşası yaratmadan doğrudan sayfa üzerinde çalışan, kullanıcıyı yönlendiren modüler paneller.



Otomatik Veri Doğrulama: Arka plan servisleri sayesinde yanlış formatta, hatalı veya eksik veri girişini anında engelleyen koruma mekanizması.



Geçmiş Ay & Arşiv Yönetimi: Geçmiş dönemlere ait vakaların ve arşivlenmiş bakım-
ların tek tıkla sorgulanabilmesi ve dinamik aktarımı.



Nesne Tabanlı (OOP) Olay Dinleme: Dinamik oluşturulan buton ve arayüz eleman-
larının sınıflar üzerinden esnek yönetimi.

Kullanıcı İçin Tavsiyeler

Sistemin kararlı, hızlı ve veritabanı bütünlüğünü bozmadan çalışabilmesi için dikkat edilmesi gereken temel kullanım ve veri yönetim kuralları aşağıda sunulmuştur:



Var Olan Şablonları Koruyun ve Bağımsız Raporlar Üretin:

ARIZA, BAKIM veya GECENAY sayfalarında yer alan mevcut veri yapılarını ve formülleri doğrudan bozmamaya özen gösteriniz.

Özel grafikler, pivot tablolar veya detaylı analizler oluşturmak istediğinizde; ilgili sayfaları kopyalayıp ****yeni bir Excel dosyasına aktararak**** çalışmanız, ana uygulamanın kararlılığını koruyacaktır.



Personel/Vaka Ekleme ve Çıkarma Kuralları:

Veritabanı sayfalarına yeni bir satır/personel ekleyeceğiniz veya çıkaracağınız zaman; **"AÇIK VAKA SAYISI"** ve **"KAPALI VAKA SAYISI"** hücrelerini ****aynı sütunda kalma koşuluyla**** serbestçe aşağı veya yukarı kaydırabilirsiniz. Sistem bu başlıkların yerini dinamik olarak tespit etmektedir.



Manuel Kullanım Esnekliği:

İstisna durumlarda (örneğin sisteme gelen raporun tarihi sistem tarafından otomatik okunamayan bozuk/farklı bir formatta ise); otomasyon panellerini kullanmak yerine veritabanı sayfalarını elle (*manuel*) doldurmaya devam edebilirsiniz.



Personel İsimlerinde Yazım Standardı:

Veritabanına yeni personel eklerken ismin başındaki/sonundaki ekstra boşluklara ve Türkçe karakter kullanımına dikkat ediniz. Raporlardaki isim ile veritabanındaki isim birebir aynı olmalıdır.



Altın Kural - Düzenli Kayıt ve Yedekleme:

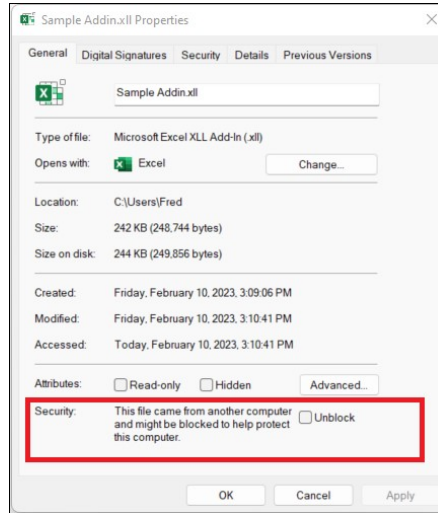
Toplu güncelleme, personel ekleme/çıkarma veya dönem sonu veri aktarımları öncesinde dosyanızın düzenli bir yedeğini almayı ve değişiklik sonrasında çalışma kitabını kaydetmeyi unutmayınız.

Sorun Giderme

Uygulamanın kullanımı sırasında karşılaşılabilecek güvenlik engellemeleri, kütüphane bağımlılıkları ve teknik aksaklıklar ile çözüm adımları aşağıda detaylandırılmıştır.

- **Güvenlik, Makro ve Antivirüs Engellemeleri:**

- **VBA Makro Engeli:** Kullanıcı dosyayı açtığında makrolar etkinleştirilmezse `Workbook_Open` tetiklenmez ve arayüz panelleri yüklenmez. Dosya açıldığında üst barda çıkan "**İçeriği Etkinleştir**" (*Enable Content*) uyarısı onaylanmalıdır.
- **MOTW (Mark of the Web) ve İnternet Engeli:** Dosya ağ üzerinden veya e-posta ile indirildiğinde Windows dosyayı engelleyebilir ve makroların çalışmasını durdurabilir. Dosya sağ tıklanıp Özellikler (*Properties*) penceresinden en alttaki "**Engellemeyi Kaldır**" (*Unblock*) kutucuğu işaretlenip Kaydet edilmelidir.



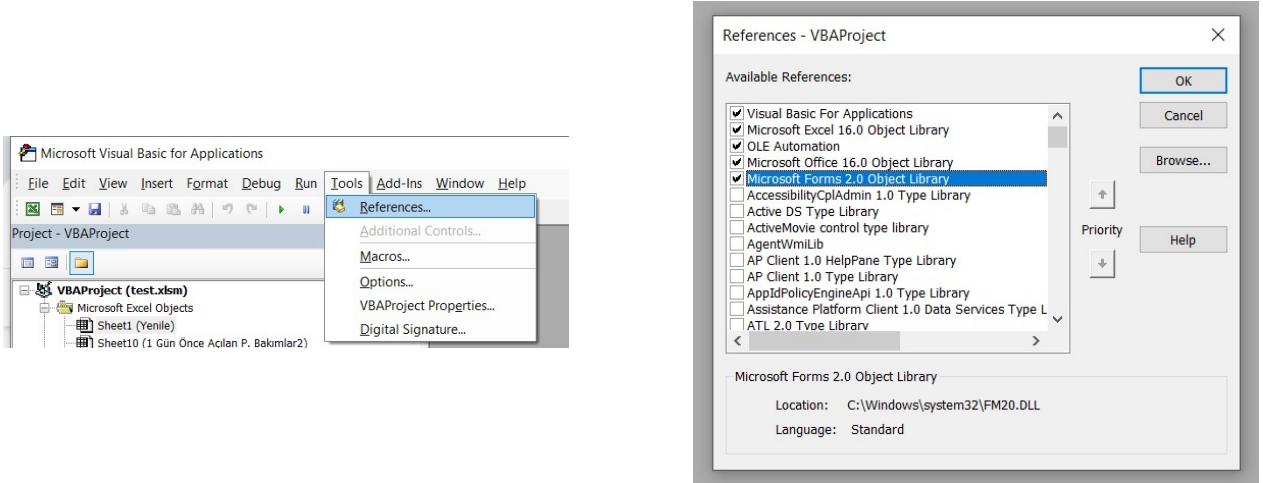
Şekil 3.1: Windows MOTW Güvenlik Engeli ve Unblock (Engellemeyi Kaldır) Seçeneği

ActiveX Denetimleri ve Güvenlik Sınırlamaları: Kod dinamik olarak `Forms.Frame.1`, `Forms.CommandButton.1` gibi OLE nesneleri oluşturur. Office güvenlik güncellemeleri veya Kill-Bit politikaları ActiveX denetimlerinin çalışma anında (*runtime*) eklenmesini engelleyebilir. Excel Trust Center (Güven Merkezi) ayarlarından ActiveX denetimlerinin engellenmediğinden emin olunmalıdır.

Kütüphane ve Eksik Bileşen (Reference) Hataları

- **Scripting.Dictionary (Windows Bağımlılığı):** Kod içerisinde `CreateObject ("Scripting.Dictionary")` kullanılmıştır. macOS (Mac için Excel) üzerinde `scrrun.dll` bulunmadığı için kod *Run-time error 429* hatası verir. Uygulama yalnızca Windows işletim sisteminde çalıştırılmalıdır.
- **MSForms (Microsoft Forms 2.0 Object Library) Bağımlılığı:** Panel nesneleri `MSForms.Frame`, `MSForms.CommandButton` tipleriyle tanımlanmıştır. Excel kurulumunda VBA desteği eksikse veya nesne kütüphanelerinde bozulma (*MISSING*) varsa "*User-defined type not defined*" hatası alırsınız.

Bu hatayı düzeltmek için klavyeden **Alt + F11** tuşlarına basarak VBA Editörüne girilmelidir. VBA penceresinde üst menüden **Tools -> References** seçeneğine tıklanmalı (Bkz. Şekil 3.2) ve açılan listede *Microsoft Forms 2.0 Object Library* kutucuğu işaretlenerek onaylanmalıdır.



Şekil 3.2: Alt + F11 ile VBA Editörüne Erişildikten Sonra Referans Ekleme Adımları (Solda: Tools -> References Menüsü, Sağda: MS Forms 2.0 Seçimi)

- **Arayüz Panellerinin Yüklenmemesi ve Manuel Yenileme:** Uygulama açılışında paneller temizlenip otomatik olarak yeniden inşa edildiği için (*Workbook_Open* tetiklenmesiyle), ilk açılışta paneller görünmezse en pratik çözüm **dosyayı kaydedip kapatmak ve tekrar açmaktır**.

Alternatif olarak; dosyayı kapatmadan panelleri yeniden yüklemek isterseniz, "**Yenile**" çalışma sayfasındayken klavyeden **Alt + F8** kısayol tuşlarına basabilir, açılan makro penceresinden `InitializeAppUI` fonksiyonunu seçerek "**Çalıştır**" (*Run*) butonuna tıklayabilirsiniz.

Uygulama Onarım Rehberi

4.1 Sayfa ve Panel Onarım Rehberi

Yenile Sayfası Kaybolduysa veya Silindiyse

Sistem dinamik panel mimarisine sahiptir. Sayfa silindiğinde sıfırlama yapmadan önce kodların varlığını doğrulamak yeterlidir.

Adım Adım Onarım Süreci:

1. **Kod Varlığını Doğrulayın:** Alt + F11 ile VBA editörüne girip modüllerin (mod_*) sol menüde yer aldığını teyit edin.
2. **Yeni Sayfa Açın:** Excel arayüzünde "Yenile" adında yeni ve boş bir çalışma sayfası oluşturun.
3. **Kaydedip Çıkın:** Dosyayı kaydedip kapatın. Yeniden açtığınızda otomasyon motoru yeni sayfayı algılayacak ve panelleri otomatik çizecektir.

ARIZA, BAKIM veya GECENAY Sayfaları Kaybolduysa

Veritabanı sayfalarından biri silindiyse veya adı bozulduysa:

1. Silinen sayfanın ismiyle (ARIZA, BAKIM veya GECENAY) birebir eşleşen yeni bir çalışma sayfası oluşturun.
2. Takip eden sayfadaki **Ekran Görüntüsü Referanslarını** inceleyerek tablo ve sütun yapısını orijinal haline getirin.
3. Sayfanın 1. satırına takip edilen tarihleri dd.mm.yyyy formatında ekleyin.
4. Dosyayı kaydedip kapatın. Sistem bir sonraki açılışta veritabanı adreslerini bu sayfa üzerinden yeniden haritalayacaktır.

4.2 Tarih ve Dönem Yönetimi

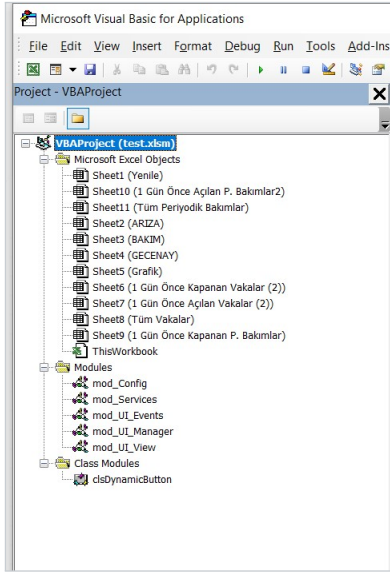
Yeni Tarih Tanımlama Mantığı

Sistem dinamik kesişim algoritması (FindIntersectCell) kullandığı için VBA kodlarında tarih güncellemesi yapmanıza gerek yoktur. ARIZA veya BAKIM sayfalarının 1. satırını yeni tarihlerle dd.mm.yyyy formatında değiştirmeniz yeterlidir.

Proje Dosya Yapısı ve Mimari

VBA Proje Ağacı

Projede yer alan tüm dosya, modül ve sınıf yapıları katmanlı mimari (*Layered Architecture*) prensiplerine uygun kurgulanmıştır.



Şekil 5.1: VBA Proje Ağacı

Excel Sheet Modülleri:

Projede yer alan **Sheet1 (Yenile)**, **Sheet2 (ARIZA)**, **Sheet3 (BAKIM)** ve **Sheet4 (GECENAY)** gibi yerleşik sayfaların içerisine kod yazılmamıştır.

Kolay okunur kod yapısı sağlamak amacıyla tüm olaylar servis katmanına yönlendirilmiştir.

ThisWorkbook:

Çalışma kitabı açılış/kapanış olaylarını (**Workbook_Open**) ve sistem ilklendirme süreçlerini yönetir.

Standart Modüller

mod_Config:

Sabitler (Constants), renk kodları, hücre adresi eşleşmeleri ve konfigürasyon değişkenlerini barındırır.

mod_Services:

Veri arama, tarih doğrulamaları ve arka plan hesaplamalarını yürüten ana iş mantığı (Business Logic) katmanıdır.

mod_UI_Events:

Kullanıcı etkileşimleri ve sayfa içi olay yönlendirmelerini dinleyen event modülüdür.

mod_UI_Manager:

Arayüz elemanlarının koordinasyonu ve oturum durumlarının yönetildiği katmandır.

mod_UI_View:

Dinamik panellerin çizimi ve görsel bileşenlerin sayfaya basılmasından sorumlu arayüz katmanıdır.

Sınıf Modülleri

clsDynamicButton:

Sayfa üzerinde çalışma zamanında (runtime) dinamik olarak oluşturulan butonların tıklanma olaylarını (Event-Handling) nesne tabanlı olarak yöneten sınıf yapısıdır.

VBA Nedir ve Temel Kavramlar

6.1 VBA (Visual Basic for Applications) Nedir?

VBA, Microsoft Office uygulamalarının arka planında çalışan, iş akışlarını otomatikleştirmek ve özel iş mantıkları (Business Logic) yürütmek için kullanılan nesne yönelimli ve olay güdümlü bir programlama dilidir. Excel başta olmak üzere Word, Outlook, Access gibi tüm Office araçlarında yerleşik olarak bulunur.

>_ Projedeki Temel Rolü

Bu projede VBA; hücre bazlı manuel işlemleri ortadan kaldırarak arka planda çalışan servis fonksiyonları üzerinden vaka tarama, tarih matrisi eşleştirme ve veritabanı kayıt süreçlerini otomatikleştirir.

6.2 Excel VBA Temel Bileşenleri

Bir Excel projesindeki VBA bileşenleri varsayılan (yerleşik) ve geliştirici tarafından eklenen özel yapılar olarak ikiye ayrılır. Projede kullanılan mimari bileşenler ve varlık durumları aşağıda özetlenmiştir:

Bileşen Türü	Varlık Durumu	Sistemdeki Görevi ve Kullanım Amacı
ThisWorkbook	Yerleşik (<i>Her Projede var</i>)	Dosya açılış/kapanış olaylarını dinler.
Sheet Modülleri	Yerleşik (<i>Her Projede var</i>)	Sayfaya özel olayları dinler. (<i>Bu projede temiz mimari için boş bırakılmıştır</i>)
Standart Modül	Özel (<i>Eklenir - .bas</i>)	Arka plan servis fonksiyonlarını ve genel hesaplama mantıklarını barındırır.
Sınıf Modülü	Özel (<i>Eklenir - .cls</i>)	Nesne tabanlı programlama (OOP) sağlar.
UserForm	Özel (<i>Eklenir - .frm</i>)	Form arayüzüdür. (<i>Bu projede UI karmaşıklığını önlemek için kullanılmamıştır</i>).

Tablo 6.1: VBA Bileşenlerinin Projedeki ve Excel'deki Yapısal Durumu

! Proje Mimarisi Notu

Bu projede kullanıcı arayüzü karmaşıklığını önlemek adına **UserForm kullanılmamış** ve kod bağımlılığını azaltmak için **Sheet modüllerine kod yazılmamıştır**. Tüm otomasyon mantığı Standart ve Class modülleri üzerinden servis odaklı yürütülmektedir.

6.3 VBA Programlama Mantığı ve Temel Yapılar

VBA (Visual Basic for Applications), nesne tabanlı ve olay güdümlü mimarisiyle performanslı otomasyonlar oluşturulmasını sağlar. Profesyonel VBA geliştirmede bellek yönetimi, değişken bildirimleri ve modüler kod tasarımı temel yapı taşlarıdır.

🛡 Katı Değişken Tanımlama (Option Explicit): Modülün en başında yer alan **Option Explicit** bildirimi, değişkenlerin önceden tanımlanmasını (**Dim**) zorunlu kılar. Bu standart; yazım hatalarından doğan mantıksal hataları (bug) engeller, yanlış veri türü kullanımının önüne geçer ve bellekte gereksiz alan tahsisini önleyerek çalışma zamanı performansını optimize eder.

📦 Nesne Hiyerarşisi (DOM) ve Bellek Yönetimi: VBA hiyerarşik bir nesne modeli kullanır (**Application** → **Workbook** → **Worksheet** → **Range**). Kod içerisinde hücre veya sayfa işlemleri yapılırken nesnelerin bir değişkene atanarak (**Set ws = ThisWorkbook.Sheets(...)**) başvurulması, Excel'in grafik arayüz yükünü azaltır ve çalışma süresini ciddi oranda kısaltır.

🔗 Prosedür (Sub) ve Fonksiyon (Function) Mimarisi: VBA'de kod blokları işlevlerine göre iki ana yapıya ayrılır:

- **Sub (Prosedür):** Belirli bir eylem dizisini çalıştırır ancak geriye bir değer döndürmez. Genellikle buton tetiklemeleri, arayüz iklendirmeleri ve akış kontrolü için kullanılır (Örn: **InitializeAppUI**).
- **Function (Fonksiyon):** Parametre alıp veri işleyen ve işlem sonucunda geriye bir değer döndüren yapılardır. Hesaplama, arama ve veri doğrulama işlemleri için tercih edilir (Örn: **GetReportDate**).

📊 Veri Tipleri ve Değişken Seçim Mantığı: VBA'de performans doğrudan seçilen veri tiplerine bağlıdır. Tam sayılar için **Long** (16-bit Integer yerine 32-bit bellek optimizasyonu), metinler için **String**, mantıksal değerler için **Boolean** ve esnek/bilinmeyen veriler için **Variant** kullanılır. Yanlış veri tipi seçimi bellek şişmesine veya işlem yavaşlığına yol açar.

⚠ Hata Yönetimi ve İstisna Kontrolü (Error Handling)

VBA'de beklenmeyen hataların uygulamayı çökertmesini önlemek için **On Error** mekanizmaları kullanılır. Riskli işlemlerden önce **On Error Resume Next** ile istisnalar es geçilip kontrol edilebilir; işlem tamamlandığında ise sistemin varsayılan hata yakalama moduna dönmesi için hemen **On Error GoTo 0** ifadesi çalıştırılmalıdır.

🔄 Ekran Yenileme ve Performans Optimizasyonu

Büyük veri kümeleriyle çalışırken ekranın her işlemde güncellenmesini engellemek için kod başında **Application.ScreenUpdating = False** ve otomatik hesaplamaları durdurmak için **Application.Calculation = xlCalculationManual** komutları kullanılır. İşlem bitiminde bu ayarların tekrar aktif edilmesi kritik önem taşır.

6.4 Projeden Örnek Kod Bloğu ve İncelemesi

Aşağıdaki kod parçacığı, projenin servis katmanından (**mod_Services**) alınmış gerçek bir fonksiyondur. Fonksiyon, kendisine verilen herhangi bir Excel sayfasının ilk satırını tarayarak geçerli rapor tarihini tespit eder:

```

1  ' Retrieves the report date from the specified worksheet header row.
2  Public Function GetReportDate(ws As Worksheet) As String
3      Dim col As Long, cellVal As Variant
4      GetReportDate = STATUS_NOT_FOUND
5      For col = 1 To 50
6          cellVal = ws.Cells(1, col).Value
7          If Not IsEmpty(cellVal) And IsDate(cellVal) Then
8              GetReportDate = Format(CDate(cellVal), "dd.mm.yyyy")
9              Exit Function
10         End If
11     Next col
12 End Function

```

Listing 6.1: Rapor Tarihi Tespit Servis Fonksiyonu (**mod_Services**)

Kodun Çalışma Mantığı ve Adım Adım Analizi

Yukarıdaki kod bloğu, modüler bir VBA servis fonksiyonunun tüm standartlarını içermektedir:

- Girdi ve Çıktı Tanımı:** **Public Function GetReportDate(ws As Worksheet) As String** satırı ile fonksiyon dışarıdan bir Excel sayfası nesnesi (**ws**) alır ve işlem sonucunda metinsel (**String**) bir tarih değeri döndürür.
- Değişken Deklarasyonu:** Döngü sayacı için **col As Long** ve hücre içeriğini tutmak için esnek veri tipi olan **cellVal As Variant** tanımlanmıştır.
- Varsayılan Durum (Fallback):** **GetReportDate = STATUS_NOT_FOUND** ifadesiyle, taramada tarih bulunamazsa fonksiyonun çökmeden güvenli bir durum döndürmesi sağlanır.
- Döngü Yönetimi ve Doğrulama:** **For col = 1 To 50** komutu ile ilk satırın 1. sütunundan 50. sütununa kadar olan hücreler taranır. **If Not IsEmpty(cellVal) And IsDate(cellVal)** koşuluyla hücrenin dolu ve geçerli bir tarih olduğu teyit edilir.
- Biçimlendirme ve Erken Çıkış:** Tarih bulunduğunda **Format(..., "dd.mm.yyyy")** ile standart biçime çevrilir ve **Exit Function** komutuyla kalan sütunlar taranmadan işlem anında bitirilir.

Genel Çalışma Mimarisi ve Modülerliğin Avantajı

Kullanıcı arayüzünde (UI katmanı) yer alan dinamik ListBox, Form ve Buton gibi tüm bileşenler verileri doğrudan hücrelerden okumak yerine bu **Servis Modüllerini (mod_Services)** çalıştırır.

Bu modüler yaklaşım sayesinde:

- İş mantığı (Business Logic) ile Görsel Arayüz (UI) birbirinden tamamen ayrılır.
- Sayfa yapısı veya Excel tasarımı değişse bile sadece servis fonksiyonu güncellenir; arayüz kodlarına dokunmak gerekmez.
- Aynı hesaplama ve tarama fonksiyonu projenin farklı yerlerinde tekrar tekrar güvenle kullanılabilir (Reusability).

Detaylı Modül İncelemeleri

7.1 ThisWorkbook

ThisWorkbook, Excel VBA mimarisinde çalışma kitabının yaşam döngüsünü (*Lifecycle*) ve global olayları (*Workbook-Level Events*) dinleyen yerleşik sınıf modülüdür. Projede uygulamanın otomatik ilklendirilmesini (*Initialization*) ve bellek değişkenlerinin hazırlanmasını sağlar.

Excel açıldığında grafik motoru, OLE bileşenleri ve Active Document yapısı henüz yüklenmemiş olabilir. Bu süreçte doğrudan makro çalıştırmak veya arayüze OLE nesneleri çizmek; grafik kaymalarına ve Korunmuş Görünüm (*Protected View*) modunda uygulamanın aniden çökmesine neden olur.

ThisWorkbook modülü, söz konusu kararsızlığı önlemek adına ****Asenkron Gecikmeli Başlatma (Asynchronous Delayed Execution)**** mimari desenini uygular:

- **Yerleşik Olay Yakalama (Workbook_Open):** Otomatik tetiklenen giriş noktasıdır (*Entry Point*).
- **İşlem Oluğu Sıfırlama (DoEvents):** Bekleyen tüm grafik yenileme ve ekran çizim olaylarını zorunlu kılar.
- **Zamanlanmış Tetikleme (Application.OnTime):** Arayüz kurma fonksiyonu olan **InitializeAppUI** prosedürünü 1 saniyelik mikro gecikmeyle Excel yürütme kuyruğuna kaydeder.

🕒 Asenkron Başlatmanın Sistem Performansına Etkisi

`Application.OnTime Now + TimeValue("00:00:01"), "InitializeAppUI"` kullanımı, dosya açılışında Excel UI motorunun tamamen stabil hale gelmesini bekler. Bu sayede servisler ve dinamik butonlar çökme riski olmadan sıfır hatayla ilklendirilir.

Aşağıdaki kod bloğu, projedeki **ThisWorkbook** sınıf modülünün tam yapısını içermektedir:

```

1  ' *****
2  ' Class: ThisWorkbook
3  ' Purpose: Standard class module that handles workbook-level events.
4  ' *****
5
6  Private Sub Workbook_Open()
7      DoEvents
8      Application.OnTime Now + TimeValue("00:00:01"), "InitializeAppUI"
9  End Sub

```

Listing 7.1: ThisWorkbook Sınıf Modülü Kod Yapısı

7.2 clsDynamicButton

clsDynamicButton, çalışma zamanında (*Runtime*) üretilen butonların tıklanma olaylarını (Click Event) Nesne Tabanlı Programlama (OOP) prensipleri doğrultusunda dinamik olarak yakalayan özel bir Sınıf Modülüdür (*Class Module*).

UserForm kullanılmayan mimarilerde arayüz elemanları doğrudan **Worksheet** üzerine çizilir. Ancak dinamik eklenen bir **MSForms.CommandButton** nesnesinin statik bir tıklama prosedürü bulunmaz. Bu kısıtlamayı aşmak adına projedeki her bir dinamik buton, clsDynamicButton sınıfından türetilen bir nesneye bağlanır ve bellek ömrünü uzatmak için global bir koleksiyonda saklanır.

Sınıf içerisindeki **WithEvents** bildirimi, buton etkileşimlerini anlık dinler. Tıklama anında izlenen işlem adımları:

- **Referans ve Kapsayıcı Tespiti:** Tıklanan buton örneği yakalanır ve üst paneli (**MSForms.Frame**) doğrulanır.
- **Olay Delegasyonu:** Panelin adına göre (**pan_Login**, **pan_OSM** vb.) tetiklenecek yönlendirme **mod_UI_Events** modülündeki handler fonksiyonlarına aktarılır.

⚠ Bellek Yönetimi ve Nesne Yaşam Ömrü (Garbage Collection)

VBA bellek yöneticisi, referansı tutulmayan sınıf örneklerini anında siler. Dinamik butonların tıklama olaylarının aktif kalması için clsDynamicButton örneklerinin **ButtonCollection** koleksiyonuna **Add** metoduyla eklenmesi kritik önem taşır.

Aşağıdaki kod bloğu, dinamik buton olaylarını dinleyen ve ilgili UI panellerine yönlendiren sınıf yapısını özetlemektedir:

```

1  '*****
2  ' Class: clsDynamicButton
3  ' Purpose: This class is used to handle events for dynamically created buttons.
4  '*****
5
6  Public WithEvents DynamicButton As MSForms.CommandButton
7
8  Private Sub DynamicButton_Click()
9      Dim parentFrame As MSForms.Frame
10
11      On Error Resume Next
12      Set parentFrame = DynamicButton.Parent
13      On Error GoTo 0
14
15      If parentFrame Is Nothing Then Exit Sub
16
17      Select Case parentFrame.Name
18          Case "pan_Login":    mod_UI_Events.HandleLoginPanelEvents DynamicButton.
                                Name, parentFrame
19          Case "pan_OSM":      mod_UI_Events.HandleOSMPanelEvents DynamicButton.Name,
                                parentFrame
20          Case "pan_Open":     mod_UI_Events.HandleOpenPanelEvents DynamicButton.Name
                                , parentFrame
21          ' . . . (pan_Closed, pan_PreviousMonth, pan_SelectPreviousMonth, . . .)
22      End Select
23 End Sub

```

Listing 7.2: clsDynamicButton Sınıf Modülü Kod Yapısı

7.3 mod_Config

mod_Config, uygulamanın tüm merkezi sabitlerini, metinsel ifadelerini, renk paletini ve veritabanı sütun eşlemelerini bünyesinde barındıran standart kod modülüdür (*Standard Module*). Kod karmaşıklığını önlemek ve bakımı kolaylaştırmak adına proje içerisindeki hiçbir fonksiyona sert kodlanmış (*Hardcoded*) metin veya sayısal değer yazılmamıştır.

Gelecekte projede yapılabilecek güncellemeler doğrudan bu modül üzerinden yönetilir:

- **Arayüz Metinleri ve Açılır Pencere (Pop-up):** Arayüzdeki buton yazıları (BTN_CAPTION), panel başlıkları (LBL_TITLE) veya kullanıcıya gösterilen tüm uyarı/bilgi bildirim mesajları (MSG_*) değiştirilmek istendiğinde yalnızca bu modüldeki ilgili sabitler güncellenir.
- **Veritabanı ve Sütun Yapısı Değişiklikleri:** İlerleyen süreçte ARIZA, BAKIM veya GECENAY sayfalarında sütun yerleri veya aranan metin bağlamaları değişirse; DB_*, CLOSED_* ve PREV_MONTH_* grubu altında tanımlanan sütun harfleri ("A", "D", "K" vb.) kolayca revize edilebilir.

Merkezi Yönetim ve Bakım Kolaylığı

Uygulamanın renk paleti (COLOR_GREEN, COLOR_RED), panel boyutları (PANEL_WIDTH) ve dinamik buton referanslarını tutan global ButtonCollection koleksiyonu bu modülde tanımlanarak projenin modülerliği garanti altına alınmıştır.

Aşağıdaki kod bloğu, mod_Config modülünün kategorize edilmiş temsili yapısını göstermektedir:

```

1  ' *****
2  ' Module: mod_Config
3  ' Purpose: Global configuration settings, constants, and variables.
4  ' *****
5
6  ' 1. SAYFA VE VERİTABANI İSİMLERİ (ARIZA, BAKIM, GECENAY)
7  Public Const MAIN_SHEET_NAME As String = "Yenile"
8  Public Const PREVIOUS_MONTH_SHEET_NAME As String = "GECENAY"
9  Public Const DATABASE_SHEET_NAME_ARIZA As String = "ARIZA"
10 Public Const DATABASE_SHEET_NAME_BAKIM As String = "BAKIM"
11
12 ' 2. VERİTABANI SÜTUN EŞLEMELERİ (COLUMN MAP)
13 Public Const DB_OSM_DATES_START_COL As String = "A"
14 Public Const DB_OPEN_CELL_COL As String = "D"
15 Public Const DB_CLOSED_NAME_COL As String = "B"
16 Public Const ALL_CASES_ASSIGNED_COL As String = "K"
17 ' . . . (Diğer sütun ve hücre adres tanımlamaları)
18
19 ' 3. POP-UP BİLDİRİM VE UYARI MESAJLARI (MESSAGES)
20 Public Const MSG_OPEN_SAVE_SUCCESS As String = "Açık kayıt sayıları veritabanına
    kaydedildi."
21 Public Const MSG_ALL_OPERATIONS_COMPLETED As String = "Tüm güncelleme işlemleri baş
    arıyla tamamlandı!"
22 ' . . . (Diğer bildirim ve hata mesajları)
23
24 ' 4. GLOBAL DEĞİŞKENLER VE ETKİLEŞİM KOLEKSİYONU
25 Public ButtonCollection As New Collection

```

Listing 7.3: mod_Config Modülü Sabit ve Yapılandırma Tanımları

7.4 mod_UI_Manager

mod_UI_Manager, dinamik panel geçişlerini, OLE nesnelerinin bellek yönetimini ve oturum kilit durumlarını orkestre eden servis katmanıdır. Sayfa üzerindeki tüm arayüz döngüsünün orkestrasyonunu üstlenir.

1. InitializeAppUI Prosedürü

Çalışma kitabı açıldığında veya oturum sıfırlandığında tetiklenen ilk ilkendirme giriş noktasıdır. Uygulama altyapısını sırayla ayağa kaldırır.

```

1 Public Sub InitializeAppUI()
2     If Not Application.ActiveProtectedViewWindow Is Nothing Then Exit Sub
3     Dim ws As Worksheet
4     On Error Resume Next
5     Set ws = ThisWorkbook.Sheets(MAIN_SHEET_NAME)
6     On Error GoTo 0
7     If ws Is Nothing Then Exit Sub
8
9     BuildNavigationSystem
10    SwitchTo "Login"
11
12    With ws
13        .ScrollArea = "A1"
14        .EnableSelection = xlNoSelection
15    End With
16 End Sub

```

Listing 7.4: InitializeAppUI Prosedürü Kod Yapısı

► Fonksiyon Özeti

Navigasyon altyapısını kurup açılış ekranı olarak **Login** panelini çağırır. Kullanıcının sayfa hücreleriyle doğrudan etkileşime girmesini engellemek adına **ScrollArea** ve seçim izinlerini kısıtlar.

2. BuildNavigationSystem Prosedürü

Tüm arayüz panellerini sıfırdan oluşturan ve bellek üzerindeki eski nesne referanslarını güvenli temizleyen yapıcı kısımdır.

```

1 Public Sub BuildNavigationSystem()
2     ' Dynamic Button Koleksiyonunu ve OLE Nesnelerini Temizleme
3     If Not ButtonCollection Is Nothing Then
4         Dim btnItem As Object
5         For Each btnItem In ButtonCollection
6             Set btnItem.DynamicButton = Nothing
7         Next btnItem
8         Set ButtonCollection = Nothing
9     End If
10    Set ButtonCollection = New Collection
11
12    ' . . . (bx_Anchor kontrolü ve eski OLE nesnelerinin temizlenmesi)
13
14    ' Panellerin mod_UI_View Üzerinden Gizilmesi
15    mod_UI_View.CreateLoginPanel ws
16    mod_UI_View.CreateOSMPanel ws
17    mod_UI_View.CreateOpenPanel ws
18    ' . . . (Closed, PreviousMonth vb. diğer panellerin oluşturulması)
19 End Sub

```

Listing 7.5: BuildNavigationSystem Prosedürü (Özet)

3. SwitchTo Prosedürü

Paneller arası görünürlük geçişini ve liste bileşenlerinin dinamik veriye bağlanmasını sağlayan ana yönlendirme fonksiyonudur.

```

1 Public Sub SwitchTo(panelName As String)
2     Dim ws As Worksheet, oleObj As OLEObject, realFrame As MSForms.Frame
3     Set ws = ThisWorkbook.Sheets(MAIN_SHEET_NAME)
4     If ws.OLEObjects("pan_" & panelName) Is Nothing Then Exit Sub
5
6     ' Tüm Panelleri Gizle ve Hedef Paneli Konumlandır
7     ' . . . (ws.OLEObjects("pan_*").Visible = False ve hedef panel hizalama)
8
9     Set realFrame = oleObj.Object
10
11     ' Panele Özel ListBox Formatlaması ve Render Yönlendirmesi
12     Select Case panelName
13         Case "OSM"
14             ' . . . (lst_Status genişlik/sütun ayarları)
15             mod_UI_View.RenderOSMSheetStatus realFrame, False
16         Case "Open"
17             ' . . . (lst_OpenStatus genişlik/sütun ayarları)
18             mod_UI_View.RenderOpenSheets realFrame
19             ' . . . (Closed, PreviousMonth vb. diğer Case yapıları)
20     End Select
21 End Sub

```

Listing 7.6: SwitchTo Prosedürü (Özet)

⚙️ Fonksiyon Özeti

Diğer servis ve event modüllerinden tek satırla çağrılarak yumuşak panel geçişi sağlar. Ek-randaki tüm panelleri gizleyip hedef paneli hizalar, **ListBox** sütun ayarlarını yapar ve ilgili **Render** prosedürünü tetikler.

4. Yardımcı Kilit Fonksiyonları (LockPanel vb.)

İş akışı sırasında geçersiz durumlarda kullanıcının hatalı adım atmasını önlemek amacıyla buton erişimlerini kısıtlayan yardımcı prosedürlerdir.

🔒 Fonksiyon Özeti

LockPanel ve **LockPreviousMonthPanel** prosedürleri; işlem adımları tamamlandığında ilgili panellerdeki "Sonraki Adım" butonlarının **Enabled** özelliğini **False** konumuna getirip renklerini griye dönüştürerek geçişleri kilitler.

📌 mod_UI_Manager Genel Özet ve Mimari Rolü:

mod_UI_Manager, arayüzün dinamik yaşam döngüsünü tamamen arka plandaki iş mantığından (*Business Logic*) ve çizim bileşenlerinden (*View Layer*) soyutlar.

Tüm panel oluşturma, temizleme, boyutlandırma ve erişim kısıtlama süreçlerini tek bir merkezde toplayarak projenin Modüler Yönetim, Bellek Güvenliği ve Kullanıcı Deneyimi (UX) standartlarını teminat altına alır.

7.5 mod_UI_Events

`mod_UI_Events`, kullanıcı arayüzündeki (UI) etkileşimleri, tıklama olaylarını (*Events*) ve form girdilerini karşılayarak arka plandaki iş mantığına (*Business Logic*) aktaran olay işleyici (*Event Handler*) servis katmanıdır.

Sınıf modülleri (`clsDynamicButton`) üzerinden gelen tıklama sinyalleri, butonun içerisinde bulunduğu üst panel bağlamına (`MSForms.Frame`) göre bu modüldeki ilgili prosedürlere aktarılır. Modül, gelen parametreleri ve mevcut uygulama durumunu değerlendirerek yönlendirme ve veri işleme adımlarını yürütür.

Modülün üstlendiği temel kritik görevler şunlardır:

- **Dinamik Akış ve Yönlendirme Yönetimi:** Kullanıcının panel geçiş taleplerinde ("İleri", "Geri") mevcut iş durumunu kontrol ederek doğru panellere geçişi sağlar.
- **Veri Güncelleme Operasyonlarının Tetiklenmesi:** "Güncelle" butonlarına basıldığında, aktif panel üzerindeki veri bileşenlerini okur ve yazma operasyonu için `mod_Services` katmanına iletir.
- **Kullanıcı Girdisi Doğrulama (Validation):** Geçmiş ay seçimi ekranlarında girilen verilerin tiplerini denetleyerek hatalı veri girişini engeller.
- **Arayüz Durum Güncellemeleri:** İşlem tamamlandığında ilgili butonların renklerini ve erişim izinlerini dinamik olarak günceller.

👉 Mimari Soyutlama ve Trafik Kontrol Rolü

`mod_UI_Events` modülü bünyesinde arayüz çizim kodlarını veya veritabanı okuma/yazma algoritmalarını barındırmaz. Sadece kullanıcı isteklerini doğru servislere ileten bir **Trafik Kontrolörü** gibi çalışır.

1. Olay Karşılama ve Akış Denetimi Mekanizması

Aşağıdaki kod parçası, panel olaylarının (`HandleLoginPanelEvents`, `HandleSelectPreviousMonthEvents`) nasıl yakalandığını ve iş akışına göre yönlendirildiğini göstermektedir:

```

1 Public Sub HandleLoginPanelEvents(buttonName As String, parentFrame As MSForms.
  Frame)
2     If buttonName = "btn_ToOSM" Then
3         Dim hasOpen As Boolean, hasClosed As Boolean
4         hasOpen = mod_Services.HasAnyOpenCase()
5         hasClosed = mod_Services.HasAnyClosedCase()
6
7         If Not hasOpen And Not hasClosed Then
8             MsgBox MSG_NO_SHEETS_TO_PROCESS, vbInformation, MSG_TITLE_INFO
9         ElseIf Not hasOpen And hasClosed Then
10            mod_UI_Manager.SwitchTo "Closed"
11        Else
12            mod_UI_Manager.SwitchTo "OSM"
13        End If
14    ElseIf buttonName = "btn_ToPreviousMonth" Then
15        ' ... Geçmiş ay tarama kontrolleri ...
16    End If
17 End Sub

```

Listing 7.7: Panel Geçiş ve Akış Kontrol Mantığı

```

1 Public Sub HandleSelectPreviousMonthEvents(buttonName As String, parentFrame As
  MSForms.Frame)
2     Dim txtYear As MSForms.TextBox, cmbMonth As MSForms.ComboBox
3
4     If buttonName = "btn_NextFromSelectMonth" Then
5         Set txtYear = parentFrame.Controls("txt_YearInput")
6         Set cmbMonth = parentFrame.Controls("cmb_MonthSelect")
7
8         If cmbMonth.ListIndex <> -1 And IsNumeric(txtYear.Text) And Len(Trim(
          txtYear.Text)) = 4 Then
9             SelectedPreviousYear = CInt(txtYear.Text)
10            SelectedPreviousMonth = cmbMonth.ListIndex + 1
11            mod_UI_Manager.SwitchTo "PrevMonthClosed"
12        Else
13            MsgBox MSG_INVALID_YEAR_INPUT, vbExclamation, MSG_TITLE_ERROR
14        End If
15    End If
16 End Sub

```

Listing 7.8: Girdi Doğrulama ve İkincil Akış Yönetimi

2. Veritabanı Yazma Tetikleme ve Görsel Güncelleme

Veri kaydetme butonlarına basıldığında çalışan özel fonksiyonlar (HandleUpdateOpen) ve işlem sonrasında buton görsellerini güncelleyen yardımcı prosedür aşağıdadır:

```

1 Private Sub HandleUpdateOpen()
2     Dim ws As Worksheet, realFrame As MSForms.Frame
3     Set ws = ThisWorkbook.Sheets(MAIN_SHEET_NAME)
4     Set realFrame = ws.OLEObjects("pan_Open").Object
5
6     If mod_Services.ExecuteDatabaseWrite(realFrame.Controls("lst_OpenStatus"),
       cellColIdx:=4, valColIdx:=3, sheetNameColIdx:=0) Then
7         SetUpdateButtonSuccess realFrame.Controls("updateOpen"), realFrame.Controls
          ("nextOpenButton")
8         MsgBox MSG_OPEN_SAVE_SUCCESS, vbInformation, MSG_TITLE_SUCCESS
9     End If
10 End Sub
11
12 Private Sub SetUpdateButtonSuccess(btnUpdate As MSForms.CommandButton, btnNext As
  MSForms.CommandButton)
13     btnUpdate.BackColor = RGB(46, 204, 113)
14     btnUpdate.Caption = BTN_CAPTION_UPDATED
15     btnUpdate.Enabled = False
16     btnNext.Enabled = True
17     btnNext.BackColor = COLOR_GREEN
18     btnNext.ForeColor = COLOR_WHITE
19 End Sub

```

Listing 7.9: Veri Yazma Tetikleme ve Buton Durum Güncelleme

i mod_UI_Events Genel Özet ve Mimari Rolü:

mod_UI_Events, arayüz ile iş katmanı arasında gevşek bağlılık (*Loose Coupling*) ilkesini uygular. Olay bazlı tetiklemeleri merkezi bir yapıda toplayarak uygulamanın bakımını kolaylaştırır, kod tekrarlarını önler ve kullanıcı etkileşimlerinin güvenli bir iş akışı çerçevesinde gerçekleşmesini teminat altına alır.

7.6 mod_UI_View

`mod_UI_View`, uygulamanın Grafik Kullanıcı Arayüzü (GUI) bileşenlerinin çalışma zamanında (*Runtime*) dinamik olarak inşasından, görsel nesne parametrelerinin yapılandırılmasından ve hesaplanan verilerin ekran elemanlarına aktarılmasından (*Rendering*) sorumlu Görünüm Katmanı (*View Layer*) modülüdür.

Uygulamada standart `UserForm` yapısı yerine, `Worksheet` üzerine gömülen asenkron OLE Panel-leri (`MSForms.Frame`) tercih edilmiştir. Bu mimari yaklaşımda `mod_UI_View`, arayüzün nesnel tasarım kalıplarına uygun şekilde soyutlanmasını sağlar. İş mantığı katmanı (`mod_Services`) ile kullanıcı etkileşim katmanı (`mod_UI_Events`) arasında köprü kurarak, ham veriyi görsel tablolara (`ListBox`) dökme işlevini üstlenir.

Modülün temel tasarım ve mimari sorumlulukları üç ana grupta toplanmaktadır:

- **Dinamik Kontrol Oluşturma (UI Builder):** Çalışma sayfasına eklenecek `Frame`, `CommandButton`, `ListBox`, `Label`, `TextBox` ve `ComboBox` gibi nesnelerin koordinat, boyut, renk ve varsayılan durumlarını programatik olarak tanımlar.
- **Veri Bağlama ve Görselleştirme (Data Rendering):** Servis katmanından dönen dizileri ve koleksiyonları dinamik listelere yükler; hata durumlarında kullanıcıya durum göstergeleri (OK, HATA) sunar.
- **Olay Dinleyici Kaydı (Event Registration):** Çizilen her bir dinamik butonu, tıklama olaylarının yakalanabilmesi amacıyla merkezi `ButtonCollection` koleksiyonuna bağlar.

Arayüz Bileşenleri ve Modüler Yapı

Görsel katmanda yer alan tüm panel oluşturma (`Create*`) ve veri işleme (`Render*`) Prosedürleri, `mod_Config` modülündeki merkezi sabitleri (`PANEL_WIDTH`, `COLOR_GREEN` vb.) referans alır. Bu sayede arayüz tasarımındaki renk ve boyut değişiklikleri kod tekrarı olmadan tek bir noktadan yönetilebilir.

1. Geliştirme Ortamı ve Modül Yapısı

Aşağıdaki ekran görüntüsünde, VBA Geliştirme Ortamı (*VBE*) üzerinde `mod_UI_View` modülünün barındırdığı yapıcı, yardımcı ve render prosedürlerinin genel listesi sunulmaktadır:

2. Arayüz Oluşturucuları (UI Builder Methods)

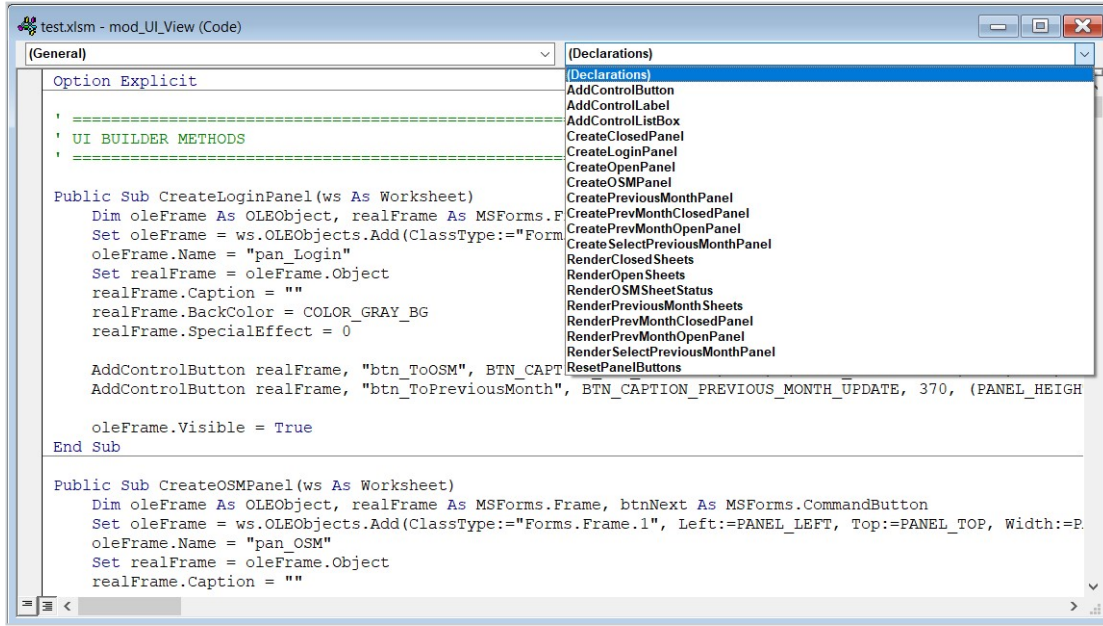
Çalışma kitabı ilk açıldığında veya yapılandırma sıfırlandığında, sistemdeki her bir iş akışı paneli (`Login`, `OSM`, `Open`, `Closed`, `PreviousMonth`, `SelectPreviousMonth` vb.) özel oluşturucu prosedürler tarafından inşa edilir.

Panel yapım aşamasında izlenen standart işlem adımları şunlardır:

1. Aktif sayfada `OLEObjects.Add` fonksiyonu ile yeni bir `MSForms.Frame` örneği açılır.
2. Panelin arka plan rengi, kenarlık stili ve konum parametreleri ayarlanır.
3. Yardımcı private metotlar (`AddControlButton`, `AddControlLabel`, `AddControlListBox`) kullanılarak panel içerisine gerekli kontrol elemanları yerleştirilir.
4. Oluşturulan buton referansları bellek kaybolmasını önlemek için `clsDynamicButton` wrapper sınıfına tescil edilir.

3. Veri Render Yöneticileri (UI Renderer Methods)

Kullanıcı paneller arasında geçiş yaptığında veya tarama/güncelleme butonlarına bastığında `Render*` prosedürleri devreye girer. Bu prosedürler, arka plandaki servislerden dönen dinamik veriyi okur ve `ListBox` elemanlarını kolon genişliklerine göre biçimlendirerek doldurur.



Şekil 7.1: mod_UI_View Modülü Prosedür ve Fonksiyon Dizilimi

Render sürecinde öne çıkan temel mantıksal mekanizmalar:

- **Sütun Başlıkları ve Tablo Hizalama:** Her render işleminde `ListBox.Clear` ile liste sıfırlanır ve ilk satıra sabit tablo başlıkları yüklenir.
- **Otomatik Veri Dönüşümü ve Eşleme:** Veritabanı hücre adresleri (D4, K2 vb.) ve ilişkili sayfa isimleri birleştirilerek açıklayıcı metinler (*Örn: D4 (ARIZA)*) halinde listeye aktarılır.
- **Zamanlayıcı ve Otomatik Kilit Mekanizması:** Ekran render edildikten sonra tarama butonlarının aktif kalması ve hatalı çift tıklamaların önlenmesi amacıyla `Application.OnTime` kullanılarak panel belirli bir süre sonra otomatik kilit durumuna alınır.
- **Boş Satır Tamamlama (Dikey Izgara Düzeni):** Görsel bütünlüğü korumak amacıyla veriler yüklendikten sonra `ListBox` dikey boyutu dolana kadar boş satır tamamlama döngüsü (`Do While ListBox.Count < 12`) çalıştırılır.

⚠ Arayüz Kararlılığı ve Hata İşleme

Render işlemleri esnasında geçersiz sayfa yapıları veya silinmiş OLE nesneleri nedeniyle çökme yaşanmaması adına, buton sıfırlayıcı yardımcı prosedürlerde (`ResetPanelButtons`) yerel hata yönetimi (`On Error Resume Next`) uygulanmaktadır.

📌 mod_UI_View Genel Özet ve Mimari Rolü:

`mod_UI_View`, uygulamanın tüm görsel bileşenlerini ve tablo ekranlarını kod üzerinden dinamik üreten bir ****Grafik Motoru**** işlevi görür. Veri işleme mantığını görsel tasarımdan tamamen ayırarak (MVC mimari yaklaşımı), projenin sürdürülebilirliğine ve kolay genişletilebilir bir arayüz yapısına kavuşmasına doğrudan katkı sağlar.

7.7 mod_Services

mod_Services, uygulamanın tüm veri işleme, hesaplama, veritabanı keşişim adresi bulma, sayfa tarama ve Excel hücre yazma algoritmalarından sorumlu ****İş Mantığı Katmanıdır (Business Logic Layer)****.

Arayüz katmanları (**mod_UI_View**, **mod_UI_Events**) kullanıcı etkileşimlerini yönetip form verilerini toplarken, **mod_Services** arka planda verinin doğruluğunu denetler, metrikleri hesaplar ve veritabanı keşişim adreslerine güvenli yazma operasyonlarını yürütür.

Modül içerisinde yüksek işlem performansı ön planda tutulmuştur. Sayfalarda tek tek hücre okumak yerine, veriler belleğe dinamik diziler (*Variant Arrays*) ve hızlı erişimli Sözlük nesneleri (*Scripting.Dictionary*) olarak alınarak işlenir.

Modülün temel fonksiyonel sorumlulukları dört ana kategoride toplanmaktadır:

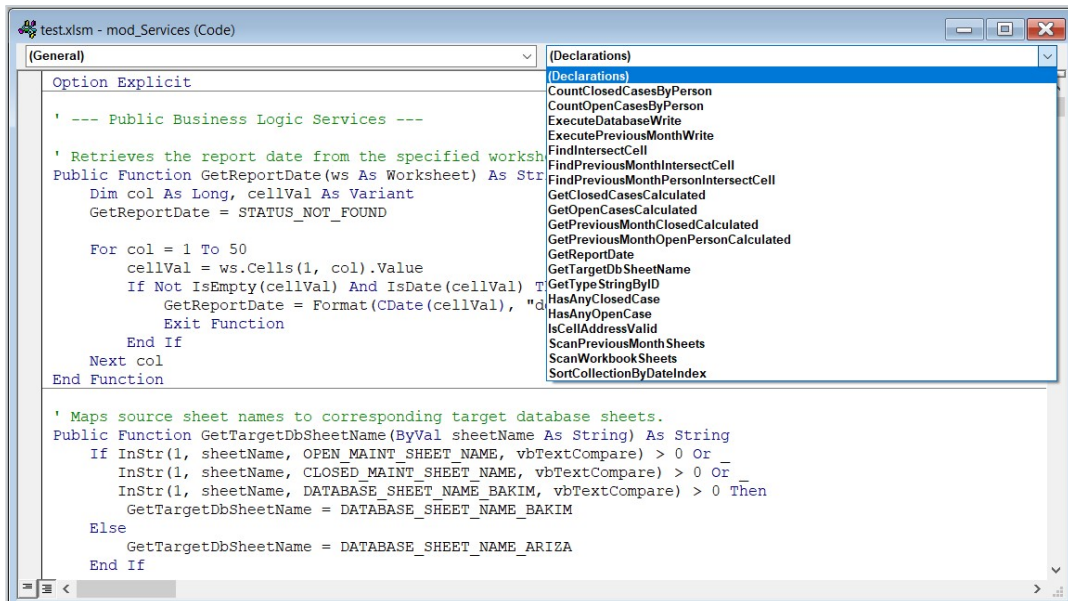
- **Çalışma Kitabı ve Sayfa Taramaları:** Kitap içerisindeki tüm sayfaları dolaşarak aktif vaka veya bakım raporlarını tespit eder (**ScanWorkbookSheets**, **ScanPreviousMonthSheets**).
- **Tarih ve Dinamik Adres Algoritmaları:** Rapor tarihlerini analiz eder, hedef veritabanı sayfasında ilgili tarih kolonu ile durum satırının keşiştiği hücreyi bulur (**GetReportDate**, **FindIntersectCell**).
- **Gelişmiş Metrik Hesaplamaları:** Personel bazlı kapalı vaka sayılarını ve geçmiş ay verilerini dinamik matrislerle hesaplar (**GetOpenCasesCalculated**, **GetClosedCasesCalculated**).
- **Veritabanı Yazma Motoru:** Arayüzdeki listelerden gelen verileri süzerek doğrudan veritabanı hücrelerine işler (**ExecuteDatabaseWrite**, **ExecutePreviousMonthWrite**).

⚙️ Performans Odaklı Bellek Yönetimi (In-Memory Data Engine)

CountClosedCasesByPerson ve **CountOpenCasesByPerson** gibi fonksiyonlar, çalışma sayfasındaki verileri **ws.Range.Value** komutuyla tek hamlede RAM üzerindeki çift boyutlu dizilere aktarır. Döngüler bellekte döndüğü için binlerce satırlık veriler milisaniyeler içerisinde işlenir.

1. Geliştirme Ortamı ve Modül Yapısı

Aşağıdaki ekran görüntüsünde, VBA Geliştirme Ortamı (*VBE*) üzerinde **mod_Services** modülünün barındırdığı iş mantığı, tarama, hesaplama ve yardımcı fonksiyonların genel dizilimi gösterilmektedir:



Şekil 7.2: mod_Services Modülü Prosedür ve Fonksiyon Dizilimi

2. Kesişim Adresi Algoritması (FindIntersectCell)

Veritabanı sayfalarında aranan tarihe ait sütun ile ilgili durum satırının kesiştiği hücre adresini (Örn: D4) dinamik olarak tespit eden temel algoritma aşağıdadır:

```

1 Public Function FindIntersectCell(ByVal dbSheetName As String, ByVal targetDateStr
  As String, ByVal searchColName As String, ByVal contextText As String) As
  String
2   Dim dbWs As Worksheet, i As Long, tRow As Long, tCol As Long
3   FindIntersectCell = STATUS_NOT_FOUND
4
5   On Error Resume Next
6   Set dbWs = ThisWorkbook.Sheets(dbSheetName)
7   On Error GoTo 0
8   If dbWs Is Nothing Or Not IsDate(targetDateStr) Then Exit Function
9
10  ' 1. Tarih Sütununu Bul
11  For i = dbWs.Columns(DB_OSM_DATES_START_COL).Column To dbWs.Columns(
    DB_OSM_DATES_END_COL).Column
12    If IsDate(dbWs.Cells(1, i).Value) Then
13      If CDate(dbWs.Cells(1, i).Value) = CDate(targetDateStr) Then tCol = i:
        Exit For
14    End If
15  Next i
16
17  ' 2. Baglam Satirini Bul (searchColName uzerinde contextText aranir)
18  ' . . . (dbLastRow hesaplanarak tRow indeksi tespit edilir)
19
20  ' 3. Kesisim Adresini Dondur (Orn: "D4")
21  If tRow > 0 And tCol > 0 Then
22    FindIntersectCell = dbWs.Cells(tRow, tCol).Address(RowAbsolute:=False,
      ColumnAbsolute:=False)
23  End If
24 End Function

```

Listing 7.10: Dinamik Kesişim Adresi Bulma Fonksiyonu

3. Personel Bazlı Gruplama ve Hesaplama Metotları

Sayfalardaki kapalı vakaları personellere göre gruplarken `Scripting.Dictionary` yapısı kullanılır. Sayılan değerler `FindIntersectCell` ile bulunan adreslerle birleştirilerek koleksiyona yüklenir:

```

1 Public Function GetClosedCasesCalculated() As Collection
2   Dim closedCol As New Collection, dict As Object, item As Variant
3   Set dict = CreateObject("Scripting.Dictionary")
4
5   For Each item In ScanWorkbookSheets()
6     If (item(1) = RPT_TYPE_CLOSED_CASE Or item(1) = RPT_TYPE_CLOSED_MAINT) And
       item(2) <> STATUS_NOT_FOUND Then
7       ' Kesisim adresi ile verileri birlestirip koleksiyona ekleme
8       dbName = GetTargetDbSheetName(CStr(item(4)))
9       For Each key In dict.Keys
10        targetCell = FindIntersectCell(dbName, CStr(item(3)),
          DB_CLOSED_NAME_COL, CStr(key))
11        If targetCell <> STATUS_NOT_FOUND And targetCell <>
          STATUS_NO_DB_SHEET Then
12          closedCol.Add Array(item(4), item(2), item(3), CStr(key), dict(
            key), targetCell)
13        End If
14      Next key
15      dict.RemoveAll
16    End If
17  Next item
18  Set GetClosedCasesCalculated = SortCollectionByDateIndex(closedCol, 3)
19 End Function

```

Listing 7.11: Kapalı Vakaların Personel Bazlı Hesaplama Mimarisi

4. Güvenli Veritabanı Yazma Motoru

Arayüzdeki listelerden okunan güncellenmiş değerler, parantezli sayfa eklerinden (Örn: D4 (ARIZA)) ayıklanarak doğrudan veritabanı hücrelerine işlenir:

```

1 Public Function ExecuteDatabaseWrite(lst As MSForms.ListBox, cellColIdx As Long,
2   valColIdx As Long, sheetNameColIdx As Long) As Boolean
3
4   Dim dbWs As Worksheet, i As Long, addr As String, caseVal As Long
5
6   For i = 1 To lst.ListCount - 1
7       addr = Trim("'" & lst.List(i, cellColIdx))
8       If InStr(addr, "(") > 0 Then addr = Trim(Split(addr, "(")(0))
9
10      If addr <> "" And addr <> STATUS_NOT_FOUND And addr <> STATUS_NO_DB_SHEET
11          Then
12              targetDbName = GetTargetDbSheetName(Trim("'" & lst.List(i,
13                sheetNameColIdx)))
14              Set dbWs = ThisWorkbook.Sheets(targetDbName)
15
16              If Not dbWs Is Nothing Then
17                  caseVal = Val("'" & lst.List(i, valColIdx))
18                  dbWs.Range(addr).Value = caseVal
19              End If
20          End If
21      Next i
22      ExecuteDatabaseWrite = True
23 End Function

```

Listing 7.12: Veritabanı Hücre Yazma Operasyonu

⚠ Hata Önleme ve Hücre Doğrulama

Yazma operasyonları öncesinde `IsCellAddressValid` fonksiyonu çalıştırılarak hücre adresinin varlığı teyit edilir. Olası sayfa bulunamama veya hatalı adres durumlarında yazma işlemi iptal edilerek kullanıcı bilgilendirilir.

📌 **mod_Services Genel Özet ve Mimari Rolü:**

`mod_Services`, uygulamanın ****Çekirdek Veri Motorudur****. Arayüz bileşenlerinden bağımsız çalışır, girdi verilerini dönüştürür, dinamik adres kesişimlerini hesaplar ve Excel hücre düzeyindeki yazma/okuma operasyonlarını yüksek performansla gerçekleştirir.

Geliştirici Rehberi

Bu bölüm, uygulamanın gelecekte yeni özelliklerle genişletilmesi, değişen kurum içi rapor yapılarına kolayca adapte edilmesi, veritabanı yapılarının güncellenmesi ve mimariye yeni dinamik panellerin eklenmesi süreçlerinde takip edilmesi gereken standartları detaylandırmaktadır.

Uygulamanın sürdürülebilir kalması için geliştiricilerin aşağıda yer alan 4 temel senaryoyu ve katmanlı mimari kurallarını adım adım uygulaması büyük önem taşımaktadır.

8.1 Senaryo 1: Gelen Rapor Biçimi veya Sütun Yapısı Değişirse

Sistemdeki en büyük mimari avantaj, kod satırlarına sert biçimde yazılmış (*Hardcoded*) sayfa/sütun isimlerinin veya dinamik metinlerin bulunmamasıdır. Kurum içi raporlarda sütun harfleri, sayfa isimleri veya vaka durum başlıkları değiştiğinde izlenecek adımlar aşağıda detaylandırılmıştır:

1. Yapılandırma Modülüne (mod_Config) Erişin:

- Klavyeden Alt + F11 kısayol tuşlarına basarak VBA Editörüne girin ve sol menüden **mod_Config** modülünü açın.
- Tüm sistem sabitlerinin kategorize edildiği bu alanda sadece ilgili sabit değerini güncelleyin.

2. Sütun Harflerini ve Hücre Adreslerini Revize Edin: Rapor sayfalarındaki sütun yerleri değiştiğinde **mod_Config** içerisindeki ilgili harfleri güncelleyin:

```
1 ' Eski: Public Const ALL_CASES_STATUS_COL As String = "AF"
2 ' Yeni Sütun Konumu:
3 Public Const ALL_CASES_STATUS_COL As String = "AG"
4 Public Const ALL_CASES_CLOSE_DATE_COL As String = "X"
```

Listing 8.1: Sütun Sabitlerinin Güncellenmesi

3. Durum Metinlerini ve Kelime Filtrelerini Genişletin: Sisteme yeni bir vaka durumu eklendiyse veya durum kelimeleri değiştiyse (Örn: "Çözüldü" yerine "Tamamlandı" veya "İşlemde" yerine "Devam Ediyor" geldiyse); sabit listelerine yeni kelimeyi virgülle ekleyin:

```
1 Public Const VALID_CLOSED_STATUSES As String = "Çözüldü,Kapandı,Kapalı,Cozuldu,
2 KAPANDI,KAPALI,Tamamlandı,TAMAMLANDI"
3 Public Const VALID_OPEN_STATUSES As String = "Açık,Atandı,Beklemede,İşlemde,Acık,
4 Atandı,Islemde,Devam Ediyor"
```

Listing 8.2: Geçerli Durum Sabitlerinin Güncellenmesi

4. İş Mantığı Katmanını Koruyun: Sütun veya durum ismi değişikliklerinde **mod_Services** içerisindeki algoritmik kodlara dokunmanıza kesinlikle gerek yoktur; arka plan servisleri dinamik olarak yenilenen bu sabitleri okuyarak kesişim hücrelerini hesaplayacaktır.

8.2 Senaryo 2: Arayüze Yeni Bir Panel Ekleme (CreateYEPYENIPanel)

Login paneline veya genel iş akışına yepyeni bir işlem ekranı (Panel) dahil edilmek istendiğinde projenin Katmanlı Mimarisi sırasıyla takip edilerek adımlar yürütülür:

1. Adım: Sabit ve Görsel Metin Tanımlamaları (mod_Config) mod_Config modülünde yeni eklenen panelin başlığı ve buton yazıları için sabitler tanımlanır:

```
1 Public Const LBL_YEPYENI_PANEL_TITLE As String = "YEPYENİ ÖZEL ANALİZ PANELİ"
2 Public Const BTN_CAPTION_TO_YEPYENI As String = "YEPYENİ ANALİZ"
```

Listing 8.3: Yeni Panel Sabitlerinin Tanımlanması

2. Adım: Panel Yapıcısı ve Çizim Prosedürü (mod_UI_View) mod_UI_View modülü içerisine yeni OLE Frame bileşenini ve içerisindeki kontrol elemanlarını (Label, Button, ListBox) çizen CreateYEPYENIPanel prosedürü yazılır:

```
1 Public Sub CreateYEPYENIPanel(ws As Worksheet)
2     Dim oleFrame As OLEObject, realFrame As MSForms.Frame
3     Set oleFrame = ws.OLEObjects.Add(ClassType:="Forms.Frame.1", Left:=PANEL_LEFT,
4         Top:=PANEL_TOP, Width:=PANEL_WIDTH, Height:=PANEL_HEIGHT)
5     oleFrame.Name = "pan_YEPYENIPanel"
6     Set realFrame = oleFrame.Object
7     realFrame.Caption = ""
8     realFrame.BackColor = COLOR_GRAY_BG
9
10    AddControllabel realFrame, "lbl_YepyeniTitle", LBL_YEPYENI_PANEL_TITLE, 40, 20
11    AddControllistBox realFrame, "lst_YepyeniStatus", 4, "150;100;120;150"
12    AddControlButton realFrame, "btn_BackToLogin", BTN_CAPTION_BACK, 40, 260, 120,
13        35, COLOR_RED
14    AddControlButton realFrame, "btn_UpdateYepyeni", BTN_CAPTION_UPDATE, 380, 260,
15        120, 35, COLOR_WHITE, &H333333
16    oleFrame.Visible = False
17 End Sub
```

Listing 8.4: CreateYEPYENIPanel Çizim Prosedürü

3. Adım: Sistem Kaydı ve Navigasyon Entegrasyonu (mod_UI_Manager)

- BuildNavigationSystem prosedürüne mod_UI_View.CreateYEPYENIPanel ws çağrısı eklenir.
- SwitchTo prosedüründeki panel gizleme alanına ws.OLEObjects("pan_YEPYENIPanel").Visible = False satırı ve Select Case bloğuna "YEPYENIPanel" seçeneği dahil edilir.

4. Adım: Olay Dinleyici ve Yönlendirme Katmanı (clsDynamicButton & mod_UI_Events)

- clsDynamicButton sınıf modülündeki Select Case parentFrame.Name kontrolüne Case "pan_YEPYENIPanel" mod_UI_Events.HandleYEPYENIPanelEvents DynamicButton.Name, parentFrame satırı eklenir.
- mod_UI_Events modülüne tıklamaları yöneten HandleYEPYENIPanelEvents prosedürü yazılır.

8.3 Senaryo 3: Yeni Bir İş Mantığı veya Hesaplama Servisi Ekleme

Uygulamaya yeni bir hesaplama, metrik analizi veya özel rapor tarama özelliği eklenmesi gerektiğinde izlenecek mimari adımlar:

1. **Servis Fonksiyonunu mod_Services İçinde Kurgulayın:** Tüm veri işleme, filtreleme ve matris kesişim adresi bulma mantığını arayüz görsel kodlarından tamamen bağımsız olarak bu modülde bir Function olarak yazın:

```
1 Public Function GetYepyeniCalculatedData() As Collection
2     Dim results As New Collection, ws As Worksheet
3     ' ... Veri tarama ve Scripting.Dictionary ile gruplama adımları ...
4     Set GetYepyeniCalculatedData = results
5 End Function
```

Listing 8.5: Yeni Hesaplama Servis Fonksiyonu Örneği

2. **Bellek İçi (In-Memory) Diziler Kullanın:** Excel hücrelerini döngü ile tek tek gezmek işlem hızını düşürür. Bunun yerine verileri tek hamlede Variant dizisine veya Scripting.Dictionary nesnesine çekerek RAM üzerinde işleyin:

```
1 Dim dataArray As Variant
2 dataArray = ws.Range("A2:Z1000").Value ' Tek seferde RAM'e alma
3 ' dataArray(i, j) üzerinden milisaniyeler içinde sorgulama
```

Listing 8.6: RAM Üzerinde Hızlı Veri İşleme

3. **Sonucu Arayüz Katmanında Render Edin:** Hesaplanan sonuçları bir Collection veya dizi olarak döndürün, ardından mod_UI_View içindeki RenderYEPYENIPanel prosedürü ile ListBox elemanına bağlayın:

```
1 Public Sub RenderYEPYENIPanel(realFrame As MSForms.Frame)
2     Dim lst As MSForms.ListBox, data As Collection, item As Variant
3     Set lst = realFrame.Controls("lst_YepyeniStatus")
4     lst.Clear
5     Set data = mod_Services.GetYepyeniCalculatedData()
6     For Each item In data
7         lst.AddItem
8         lst.List(lst.ListCount - 1, 0) = item(0)
9     Next item
10 End Sub
```

Listing 8.7: ListBox Veri Bağlama (Rendering)

Servis Mimarisinin Avantajı

İş mantığı servis fonksiyonunda kurgulandığı için, gelecekte Excel arayüzü değişse bile GetYepyeniCalculatedData fonksiyonuna dokunulmadan farklı mecralarda tekrar kullanılabilir (Reusability).

8.4 Senaryo 4: Yeni Veritabanı Çalışma Sayfası Ekleme ve Geliştirme Kuralları

Sisteme ARIZA ve BAKIM dışında örneğin KALITE veya LOJISTIK adında 3. bir veritabanı sayfası ekleneceği zaman izlenecek süreçler ve uyulması gereken katı geliştirme kuralları şunlardır:

1. Veritabanı Eşleme Fonksiyonunu Güncelleyin: `mod_Services` modülündeki `GetTargetDbSheetName` fonksiyonuna yeni sayfa koşulunu ekleyin:

```
1 Public Function GetTargetDbSheetName(ByVal sheetName As String) As String
2     If InStr(1, sheetName, "KALITE", vbTextCompare) > 0 Then
3         GetTargetDbSheetName = "KALITE"
4     ElseIf InStr(1, sheetName, "BAKIM", vbTextCompare) > 0 Then
5         GetTargetDbSheetName = DATABASE_SHEET_NAME_BAKIM
6     Else
7         GetTargetDbSheetName = DATABASE_SHEET_NAME_ARIZA
8     End If
9 End Function
```

Listing 8.8: Hedef Veritabanı Eşleme Güncellemesi

2. Yeni Veritabanı Sayfa Yapısını Oluşturun:

- Sayfanın 1. satırına takip edilecek tarihleri dd.mm.yyyy formatında yerleştirin.
- B sütununa (B kolonu) personel/vaka isimlerini yazın.
- "AÇIK VAKA SAYISI" ve "KAPALI VAKA SAYISI" arama başlıklarını ilgili satırlara ekleyin.

⚠ Katı Kodlama Standartları ve Kalite Kuralları

Geliştiricilerin kod tabanına müdahale ederken uymakla yükümlü olduğu temel kurallar:

- **Değişken Bildirimi:** Eklenen her yeni modülün ve sınıfın en başında mutlaka `Option Explicit` ifadesi yer almalıdır.
- **Hücre Bazlı Döngü Yasağı:** Çalışma sayfası hücreleri üzerinde tek tek `For Each cell In Range` döngüsü kurulmamalı; veriler belleğe (`Variant Array / Scripting.Dictionary`) alınarak milisaniyeler seviyesinde işlenmelidir.
- **Bellek Güvenliği:** Dinamik oluşturulan her bir `MSForms.CommandButton` nesnesi, bellek kayıplarını ve tıklama hatalarını önlemek için merkezi `ButtonCollection` koleksiyonuna eklenmelidir.

📌 Özet ve Katkı Yaklaşımı:

Bu proje, Modüler Katmanlı Mimari (Layered Architecture) ve MVC (Model-View-Controller) prensipleri doğrultusunda inşa edilmiştir. Yapılacak tüm geliştirmelerde iş mantığı (`mod_Services`), görünüm (`mod_UI_View`) ve olay yönlendirme (`mod_UI_Events`) katmanlarının kesinlikle birbirinden ayrık tutulması projenin sürdürülebilirliği için şarttır.

Sonuç

başak abla selam